



S.P.M College, Udantpuri

Bachelor Of Computer Application (BCA)

Part -1

– Hira Kumar

Functions in C

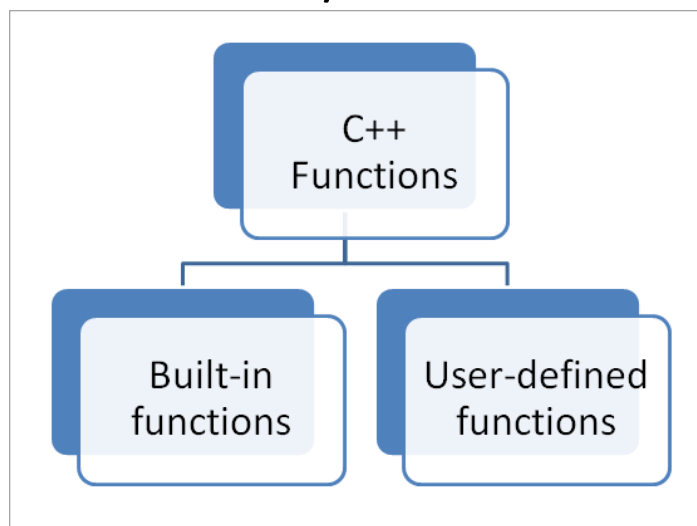
- A function is a group of statements that together perform a task. Every program in C has main() function by default. C standard library provides lot of builtin functions that you can call or use in your program.

OR

- A function is a block of code that performs a specific task and can be reuse anywhere in the program.

There are two types of function:

1. Standard Library Functions / Predefined Function / Built in Functions
2. User-defined Function / Custom Function



1. Predefine Function

- Predefine functions are the functions that are already written and stored in c standard libraries. Programmers can directly use them in their program without writing their definition.
- You only need to include the correct header file to use them.

OR

- A predefined function in C is a function that is already define in the C standard library and can be used directly by including the appropriate. header file.

2. User define Function

- A User defined function in C is a function that is define by the programmer to perform a specific task and can be called whenever needed in the program.

❖ Header Files

- Contains a collection of function prototypes, constant and preprocessor definitions
- Named with extension .h

A header file may be included in one of the two ways.

include <stdio.h> (use angle bracket <>)or
include "stdio.h". (use double quotes " ")

- ✓ The header file in angle brackets means that file reside in standard include directory (predefined files).

e.g.- #include <stdio.h>
 #include <cmath.h>
 #include <cstring.h>,.....

- ✓ The header file in double quotes means that file resides in current directory(User defined files).

e.g. - #include "hira.h"
 #include "Bihar.h",.....

❖ Preprocessor

- Preprocessor commands begin with a # and do process of instruction available in #.

e.g. #include – includes a named file
 #define – defines a (text replacement) macro

`#ifdef/#else/#endif` – conditional compilation

❖ **# define** = Often used to define constants.

e.g. `#include <stdio>`

`#define PI 3.14` //define constant & PI (identifier) is a macro

`# const float PI = 3.14;` // define constant

Q. Create a user define header file.

Step 1 create a new file save as .h extension

Example – hira.h

Write this code

```
#define NAME "Hira Kumar"
```

```
void hello();  
void hello()  
{  
printf("%s",NAME);}
```

Step 2 : Create a new file with extension .c

```
#include "hira.h"
```

```
int main()  
{  
hello();  
return 0;  
}
```

Note : both file (.h & .c) in same folder

Output : Hira kumar

❖ List of Predefine Function header files in c and its use/work

1. <stdio.h>

Used for input and output operations in C.

name	Function Use
printf()	Display output on screen {printf→it is standard output stream i.e Object, not function}
scanf()	Takes input form keyboard (no space read)
gets()	Read a string (space read – but not safe)
fgets()	Read a string (space read – safe)
puts()	Display a string
getchar()	Reads a single character
putchar()	Displays a single character
fopen()	Opens a file
fclose()	Closes a file

❖ Difference b/w gets() & fgets()

gets()	fgets()
• gets() is unsafe because it does not check input size and may cause buffer overflow. • gets() removes the newline character automatically. • gets() has been removed from modern C standards. • Syntax - gets(str);	• fgets() is safe because it reads limited characters and prevents buffer overflow. • fgets() stores the newline (\n) in the string. • fgets() is recommended and widely used. • Syntax - fgets(str, size, stdin);

Q. Program in C use all function in stdio .h

```
#include<stdio.h>
```

```
#include<math.h>
```

```

#include<string.h>
int main()
{
    char s[20];
    printf("Enter your name :");
    scanf("%s",s);
    printf("Your name is = %s \n",s);
    while(getchar() != '\n'); // clean buffer
    printf("Enter your name :");
    fgets(s,sizeof(s),stdin);
    printf("your city name is = ");
    puts(s);
    char c;
    printf("enter any character =");
    c = getchar();
    printf("your character is :");
    putchar(c);

    getch();
}

```

The image shows a C program and its execution. The code on the left is a C program that prompts the user for their name, city, and a character. The execution output on the right shows the program running and the user providing input: 'hira kumar' for the name, 'Bihar sharif' for the city, and 'H' for the character.

```

hello.c x
#include<string.h>
int main()
{
    char s[20];
    printf("Enter your name :");
    scanf("%s",s);
    printf("Your name is = %s \n",s);
    while(getchar() != '\n'); // clean buffer
    printf("Enter your name :");
    fgets(s,sizeof(s),stdin);
    printf("your city name is = ");
    puts(s);
    char c;
    printf("enter any character =");
    c = getchar();
    printf("your character is :");
    putchar(c);

    getch();
}

```

C:\Users\hira\Desktop\hello.exe

```

Enter your name :hira kumar
Your name is = hira
Enter your name :Bihar sharif
your city name is = Bihar sharif

enter any character =H
your character is :H

```